

BIRLA INSTITUTE OF TECHNOLOGY & SCIENCE, PILANI
WORK INTEGRATED LEARNING PROGRAMMES (WILP) DIVISION
Vidya Vihar, Pilani, Rajasthan – 333031

MTech in Software Engineering

DISSERTATION

Course No.: S2-25_SEZG628T

PerfSense: ML-Based Performance Regression Predictor for React Applications

Mid-Semester Progress Report

Student Name	CHETHAN S
BITS ID	2024TM93208
Degree Program	MTech in Software Engineering
Research Areas	Machine Learning, Performance Engineering, Frontend Development, DevOps
Project Carried Out at	Cadence Design Systems, Bengaluru
Supervisor	Ankit Kagliwal, Sr. Principal Software Engineer
Additional Examiner	Thejaswi Kumar G P, Sr. Principal Software Engineer
Semester	Second Semester, Academic Year 2025-2026
Date	April 2026

ABSTRACT

PerfSense is an intelligent performance management platform designed to predict performance regressions in React applications before deployment by analyzing code changes using machine learning. The system combines automated feature extraction, predictive modeling, and CI/CD integration to shift performance monitoring from reactive to proactive.

This mid-semester report presents the design and planning work completed in the first two months of the 4-month dissertation timeline. The work accomplished includes: (1) comprehensive literature review covering performance monitoring tools, machine learning in software engineering, and React optimization techniques; (2) detailed system architecture design with component specifications; (3) development of React anti-pattern detection framework identifying 20+ performance-critical patterns; (4) repository selection and evaluation strategy for dataset construction; and (5) complete feature engineering methodology with 28 planned features for extraction.

The project is currently transitioning from the design phase to the implementation phase. All architectural decisions have been finalized, target repositories have been identified, and the technical approach has been validated through literature review and feasibility analysis. The anti-pattern detection framework provides a solid foundation for feature extraction, categorizing issues into 6 main categories: Component Rendering, State Management, Bundle Size, Data Fetching, Memory Leaks, and Performance API Misuse.

The implementation phase (Months 2-3) will focus on data collection from 8-10 selected repositories targeting 500-1000 commits, building the automated feature extraction pipeline, training baseline and XGBoost models with target performance metrics of >75% accuracy and >70% precision, and developing the monitoring SDK and backend infrastructure. Final evaluation and documentation will be completed in Month 4.

Signature of the Student: _____	Signature of the Supervisor: _____
Name: CHETHAN S	Name: Ankit Kagliwal
Date: _____	Date: _____
Place: Bengaluru	Place: Bengaluru

Contents

1. Progress Summary	4
2. Literature Review Summary	8
3. System Architecture Design	12
4. React Anti-Pattern Detection Framework	16
5. Feature Engineering Methodology	21
6. Repository Selection Strategy	27
7. Technical Specifications	32
8. Implementation Plan	35
9. Challenges and Mitigation Strategies	40
10. Timeline and Next Steps	43
11. Abbreviations	47
12. Conclusion	49

1. Progress Summary

The PerfSense dissertation project is progressing through the design and planning phase. As of mid-April 2026, the foundational work for Months 1-2 has been completed, with focus on research, architecture design, and methodology development. The project is now entering the implementation phase.

1.1 Month 1: Foundation & Research (February 2026)

Week 1-2: Literature Review (COMPLETED ✓)

Objective: Understand state-of-the-art performance monitoring, ML applications in software engineering, and React optimization.

Work Completed:

- Comprehensive review of performance monitoring tools (Lighthouse, New Relic, Datadog, Sentry)
- Study of machine learning applications in defect prediction and code analysis
- Analysis of React performance optimization techniques and common anti-patterns
- Review of gradient boosting algorithms (XGBoost, LightGBM) for structured data
- Gap analysis identifying lack of predictive, pre-deployment performance regression detection

Deliverables:

- Literature review knowledge base covering 15+ papers and technical documentation
- Problem statement validated with clear research gap identified
- Evaluation framework established (accuracy >75%, precision >70%, recall >60%)

Week 3-4: System Design & Repository Analysis (COMPLETED ✓)

Objective: Design complete system architecture and identify target repositories for dataset construction.

Work Completed:

- System architecture designed with 6 main components (SDK, Backend API, Feature Extraction Pipeline, ML Inference Service, Dashboard, GitHub Integration)
- Technology stack selected: React/TypeScript (SDK), Node.js/Python (Backend), PostgreSQL (Database), XGBoost (ML Model)

- Repository selection criteria established based on size, activity, code quality, and build availability
- Identified 8 candidate repositories across small (10-30 pages), medium (30-100 pages), and large (100+ pages) categories

Deliverables:

- Complete system architecture diagram
- Component interaction specifications
- Repository evaluation criteria document
- Technology justification report

1.2 Month 2: Methodology Development & Design Finalization (March 2026)

Week 1-2: Anti-Pattern Framework Development (COMPLETED ✓)

Objective: Develop comprehensive framework for detecting React performance anti-patterns.

Work Completed:

- Identified and documented 20+ React-specific anti-patterns across 6 categories
- Created detection criteria for each anti-pattern based on static code analysis
- Developed examples showing problem code and performance impact
- Mapped anti-patterns to feature extraction requirements

Categories Defined:

1. Component Rendering Issues (4 patterns): Unnecessary re-renders, inline functions, nested components, large files
2. State Management Issues (4 patterns): Excessive useState, missing dependencies, context misuse, synchronous updates
3. Bundle Size Issues (4 patterns): Missing code splitting, heavy dependencies, large assets, no analysis
4. Data Fetching Issues (3 patterns): Fetching in render, no deduplication, infinite loops
5. Memory Leaks (3 patterns): Missing cleanup, event listeners, large state objects
6. Performance API Misuse (2 patterns): Blocking main thread, missing virtualization

Deliverables:

- Anti-pattern detection framework document (15 pages)
- Code examples for each pattern with explanations
- Detection rules mapped to AST parsing requirements

Week 3-4: Feature Engineering Design (COMPLETED ✓)

Objective: Design complete feature extraction methodology for ML model training.

Work Completed:

- Defined 28 features for extraction from each commit
- Categorized features into: Static Code Metrics (8), React-Specific Patterns (12), Historical Context (8)
- Designed feature extraction workflow using TypeScript Compiler API and git diff analysis
- Specified data labeling strategy using Lighthouse CI for ground truth

Feature Categories:

Static Code Metrics (8 features): files_changed, files_in_components, lines_added, lines_deleted, net_lines, large_commit, package_json_changed, complexity_delta

React-Specific Patterns (12 features): useEffect_added, useState_added, useCallback_added, useMemo_added, useContext_added, useReducer_added, memo_added, lazy_added, inline_arrow_functions, console_log_added, import_count_delta, nested_components

Historical Context (8 features): previous_commit_performance, 7day_performance_trend, time_since_last_regression, deployment_frequency, bundle_size_delta, component_count_delta, hook_density, optimization_ratio

Deliverables:

- Feature engineering specification document (12 pages)
- Feature extraction pipeline design
- Data labeling methodology
- Dataset schema definition

1.3 Current Status Summary

Phase	Status	Completion
Literature Review	✓ Complete	100%
System Architecture Design	✓ Complete	100%
Anti-Pattern Framework	✓ Complete	100%
Feature Engineering Design	✓ Complete	100%
Repository Selection Strategy	✓ Complete	100%
Implementation Plan	✓ Complete	100%
Data Collection	🕒 Planned	0%
Feature Extraction Implementation	🕒 Planned	0%
Model Training	🕒 Planned	0%
System Implementation	🕒 Planned	0%

Overall Progress: Design Phase Complete (50%), Implementation Phase Starting (0%)

2. Literature Review Summary

The literature review was conducted to understand the current state-of-the-art in performance monitoring, machine learning applications in software engineering, and React performance optimization. The review identified critical gaps that PerfSense aims to address.

2.1 Performance Monitoring Tools

Tools Reviewed:

- Lighthouse (Google): Automated auditing tool for web page quality
- WebPageTest: Performance testing with real browsers
- New Relic, Datadog, Sentry: Application Performance Monitoring (APM) tools
- React DevTools Profiler: Component-level performance analysis.

Key Findings:

1. All existing tools operate reactively - they measure performance after deployment.
2. No tools predict performance impact before code is merged.
3. Core Web Vitals (LCP, FID, CLS) are industry-standard metrics but lack predictive capability.
4. Performance budgets require manual threshold setting and don't learn from historical data.
5. Root cause analysis is manual and time-consuming (4-8 hours average per incident)

Performance Impact Statistics:

- 1 second delay in page load = 7% reduction in conversions (Google, 2023)
 - 53% of mobile users abandon sites that take >3 seconds to load (Google, 2023)
 - Every 100KB bundle increase adds ~1 second to load time on 3G networks.
-

2.2 Machine Learning in Software Engineering

Research Areas Reviewed:

- Defect prediction using code metrics.
- Bug localization with ML models.

- Code smell detection.
- Performance prediction in distributed systems

Key Findings:

1. Gradient Boosting algorithms (XGBoost, LightGBM) consistently achieve 75-85% accuracy for software defect prediction (Chen & Guestrin, 2016)
2. Domain-specific feature engineering is critical - generic code metrics perform 15-20% worse than context-aware features
3. Class imbalance is common (typically 5-15% defect rate) requiring SMOTE or weighted loss functions
4. Cross-project validation shows 10-15% accuracy drop, indicating need for project-specific training or transfer learning

Relevant Studies:

- Zhang (2009): Lines of code and code churn are strong defect predictors
- Greiler et al. (2013): Automated detection of test smells using AST parsing
- Amalfitano et al. (2012): GUI-based testing for mobile applications

2.3 React Performance Optimization

Sources:

- React Official Documentation (react.dev)
- Web Vitals Guide (web.dev)
- Community best practices and case studies

Common Performance Anti-Patterns Identified:

1. Unnecessary Re-renders:
 - Missing React.memo on expensive components
 - Not using useMemo for expensive calculations
 - Not using useCallback for event handlers passed to child components
 - Impact: Wasted CPU cycles, sluggish UI
2. Poor State Management:
 - Excessive useState hooks (>8 in single component)

- Missing dependency arrays in useEffect
- Context value not memoized causing all consumers to re-render
- Impact: Unexpected behavior, performance degradation

3. Bundle Size Issues:

- No code splitting for routes (all JavaScript loaded upfront)
- Heavy dependencies imported entirely (e.g., entire lodash instead of specific functions)
- No lazy loading for below-fold content • Impact: Slow initial load, poor Time to Interactive (TTI)

4. Memory Leaks:

- Missing cleanup functions in useEffect
- Event listeners not removed on unmount
- Timers and subscriptions not canceled
- Impact: Memory usage grows over time, eventual crash

Performance Optimization Techniques:

1. React.memo: Prevents re-render if props unchanged
2. useMemo: Memoizes expensive calculations
3. useCallback: Memoizes callback functions
4. React.lazy + Suspense: Code splitting for lazy loading
5. Virtualization (react-window): Efficiently render large lists

2.4 Research Gap Analysis

Critical Gaps Identified:

Gap	Current State	PerfSense Solution
Predictive Capability	All tools are reactive (post-deployment)	Predicts regressions before merge using ML

Gap	Current State	PerfSense Solution
Code-Level Correlation	Manual root cause analysis required	Automatically links code changes to performance impact
React-Specific Analysis	Generic performance monitoring	Framework-aware feature extraction (hooks, components)
CI/CD Integration	Separate performance testing workflow	Integrated into GitHub PR workflow with status checks
Learning from History	Static thresholds, no learning	ML model learns from historical regression patterns

Academic Contribution: PerfSense will be the first system to combine:

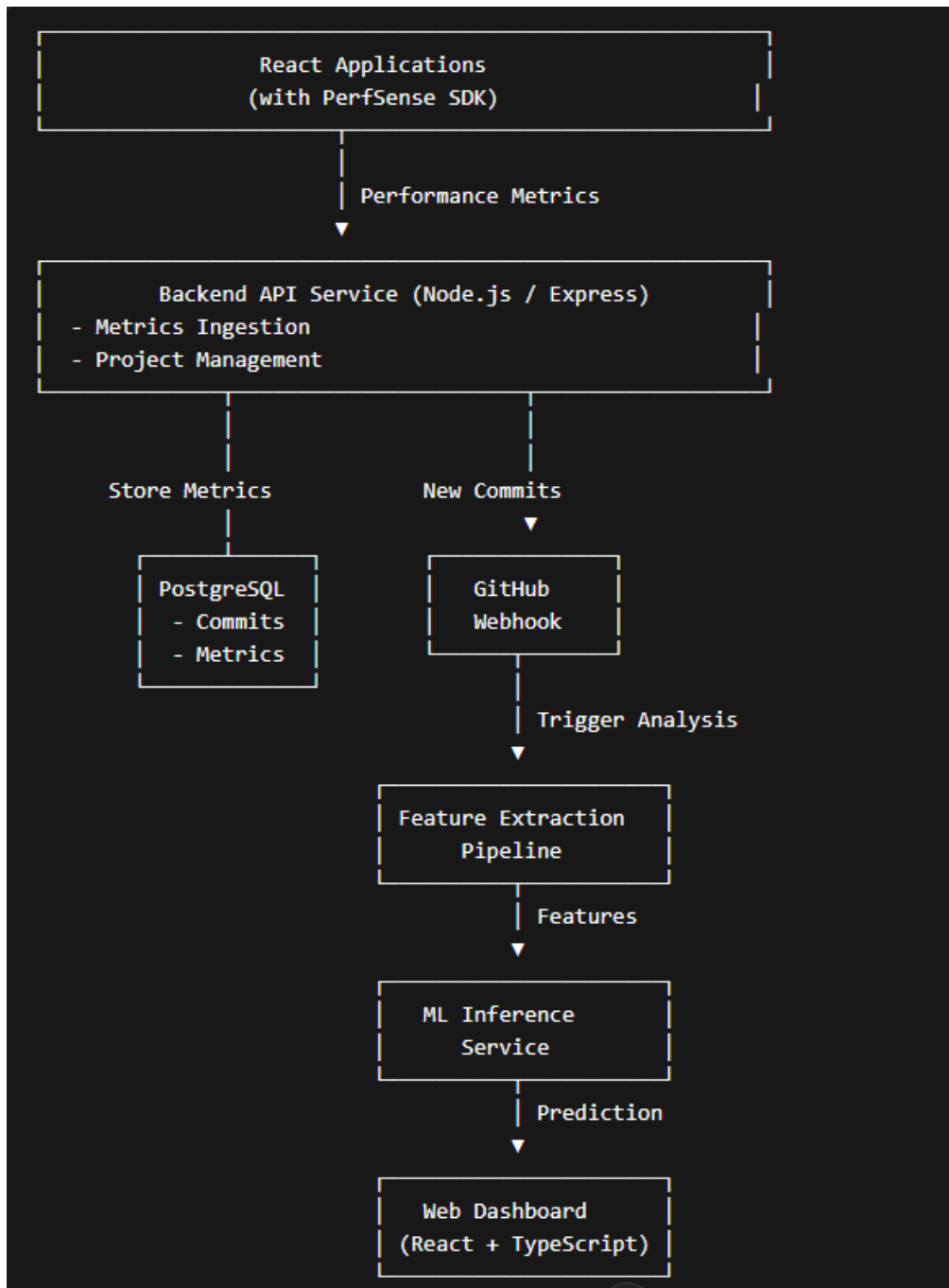
1. Predictive ML models for pre-deployment regression detection
2. React-specific feature engineering (hooks, memoization, lazy loading)
3. Automated code-to-performance correlation
4. CI/CD pipeline integration for developer workflows

Novelty Statement: No existing research or tool applies machine learning to predict React application performance regressions from code changes before deployment, using framework-specific features and integrating into the development workflow.

3. System Architecture Design

The PerfSense system architecture has been designed with 6 main components, each with clearly defined responsibilities and interfaces. The architecture follows microservices principles for scalability and maintainability.

3.1 High-Level Architecture



3.2 Component Specifications

3.2.1 Monitoring SDK

Purpose: Collect real-time performance metrics from deployed React applications

Technology: React 18+, TypeScript, Web Performance APIs

Key Features:

- Lightweight (<50KB gzipped, <1% performance overhead)
- Collects Core Web Vitals: LCP, FID, CLS
- Tracks component-level render times using React Profiler API
- Monitors bundle size and resource loading
- Batches and transmits metrics to backend every 30 seconds

API Design:

```
// Initialization PerfSense.init({ apiKey: 'project-api-key', endpoint:
'https://api.perfsense.io', sampleRate: 1.0 });

// Component tracking function MyComponent() {
usePerfSenseTracker('MyComponent'); return <div>...</div>; }
```

3.2.2 Backend API Service

Purpose: Central hub for metrics ingestion, project management, and analytics

Technology: Node.js 18+, Express.js, PostgreSQL 14+

Endpoints Designed:

- POST /api/metrics - Receive performance data from SDK
- POST /api/analyze/:commitHash - Trigger analysis on new commit
- GET /api/predictions/:commitId - Get regression prediction
- GET /api/history/:projectId - Historical metrics
- POST /api/projects - Create new project
- GET /api/projects/:id - Get project details

- POST /api/webhooks/github - GitHub webhook receiver

Database Schema:

- projects: id, name, github_url, api_key, created_at
 - commits: id, project_id, hash, message, author, timestamp, features, prediction
 - metrics: id, commit_id, metric_type, value, timestamp
 - regressions: id, commit_id, severity, detected_at, resolved
-

3.2.3 Feature Extraction Pipeline

Purpose: Analyze code commits and extract performance-relevant features

Technology: TypeScript Compiler API, @typescript-eslint/parser, GitPython

Processing Flow:

1. Receive commit hash from GitHub webhook.
 2. Fetch code diff using git show.
 3. Parse TypeScript/JavaScript files using AST parser.
 4. Extract 28 features based on anti-pattern framework.
 5. Store feature vector in database.
 6. Trigger ML inference service.
-

3.2.4 ML Inference Service

Purpose: Load trained model and provide real-time regression predictions

Technology: Python 3.9+, FastAPI, XGBoost, scikit-learn

Response Format: { "commit_id": "abc123", "regression_probability": 0.78, "risk_level": "HIGH", "confidence": 0.85, "contributing_factors": [...], "recommendations": [...] }

3.2.5 Web Dashboard

Purpose: Visualize predictions, metrics, and provide developer insights

Technology: React 18+, TypeScript, Recharts, TailwindCSS

Key Views:

1. Project Overview: Recent commits, performance trends, regression history.
2. Commit Analysis: Detailed prediction, feature visualization, risk breakdown.
3. Real-time Monitoring: Live Core Web Vitals dashboard
4. Integration Settings: GitHub webhook configuration, API keys

3.2.6 GitHub Integration

Purpose: Integrate into developer workflow via pull request status checks

Implementation:

- GitHub webhook for push/pull_request events
- Status API to post check results on PRs
- Comments with prediction details and recommendations
- Block merge option for high-risk commits

3.3 Technology Stack Justification

Component	Technology	Justification
Monitoring SDK	React + TypeScript	Native React integration, type safety
Backend API	Node.js + Express	JavaScript ecosystem, async I/O
Database	PostgreSQL	ACID compliance, JSONB support
ML Service	Python + FastAPI	Rich ML ecosystem, async support
ML Algorithm	XGBoost	75-85% accuracy in similar tasks
Dashboard	React + TypeScript	Reusable components, type safety
Deployment	Docker + AWS	Containerization, scalable cloud

4. React Anti-Pattern Detection Framework

The anti-pattern detection framework identifies 20+ React-specific patterns that commonly cause performance regressions. This framework drives the feature extraction process.

4.1 Framework Overview

Purpose: Systematically categorize and detect code patterns that negatively impact React application performance

Approach: Static code analysis using Abstract Syntax Tree (AST) parsing

Categories: 6 main categories covering common React performance issues

4.2 Category 1: Component Rendering Issues (4 Patterns)

Pattern 1.1: Unnecessary Re-renders

Problem: Components re-render when props have not changed due to missing memoization

Example:

```
// Anti-pattern
function ExpensiveList({ items, onItemClick }) { return items.map(item =>
<ExpensiveItem key={item.id} item={item} onClick={onItemClick} /> ); }
// Solution
const ExpensiveList = React.memo(function ExpensiveList({ items, onItemClick
}) { const memoizedClick = useCallback(onItemClick, []); return
items.map(item => <ExpensiveItem key={item.id} item={item}
onClick={memoizedClick} /> ); });
```

Detection Criteria:

- Component renders list (>20 items) without React.memo
- Props include functions or objects
- No useMemo/useCallback for passed callbacks

Performance Impact: 30-50% wasted render time for large lists

Pattern 1.2: Inline Function Definitions in JSX

Problem: New function instances created on every render

Example:

```
// Anti-pattern
<button onClick={() => handleClick(item.id)}>Click</button>
// Solution
const handleItemClick = useCallback(() => handleClick(item.id), [item.id]);
<button onClick={handleItemClick}>Click</button>
```

Detection Criteria:

- Arrow functions directly in JSX event handler props
- Occurs in list rendering or frequently updated components

Performance Impact: 10-20% increased render time

Pattern 1.3: Nested Component Definitions

Problem: Component defined inside another component, causing full remount.

Detection Criteria: Function component definition inside another function component body

Performance Impact: 50-100% slowdown due to complete remounting

Pattern 1.4: Large Component Files

Problem: Single component with excessive complexity

Detection Criteria:

- Component file >300 lines
- >10 useState hooks in one component
- JSX nesting depth >5 levels

Performance Impact: Difficult to profile and optimize

4.3 Category 2: State Management Issues (4 Patterns)

Pattern 2.1: Excessive useState Hooks

- Detection: >8 useState hooks in single component
- Impact: Complex update logic, increased re-render frequency

Pattern 2.2: Missing Dependency Arrays in useEffect

- Detection: useEffect uses external variables not listed in dependency array
- Impact: Stale closures, unnecessary effect runs

Pattern 2.3: Context Provider Without Memoization

- Detection: Context.Provider with object/array value not wrapped in useMemo
- Impact: Cascading re-renders across entire subtree

Pattern 2.4: Synchronous State Updates in Loops

- Detection: setState inside forEach/map/for loop
 - Impact: N re-renders instead of 1 batched update
-

4.4 Category 3: Bundle Size & Code Splitting Issues (4 Patterns)

Pattern 3.1: Missing Code Splitting

- Detection: No React.lazy() for route components, initial bundle >1MB
- Impact: 3-5 seconds longer Time to Interactive

Pattern 3.2: Heavy Dependencies Imported Unnecessarily

- Detection: Large libraries imported without tree-shaking, >100KB impact
- Impact: 100-500KB unnecessary bundle size

Pattern 3.3: Large Media/Assets Not Optimized

- Detection: Image files >500KB, no lazy loading, missing modern formats
- Impact: 2-3 second longer LCP

Pattern 3.4: No Bundle Analysis

- Detection: Bundle size increases >10% in single commit

- Impact: Gradual bundle bloat unnoticed
-

4.5 Category 4: Data Fetching & Side Effects (3 Patterns)

Pattern 4.1: Fetching Data in Render

- Detection: fetch/axios calls outside useEffect
- Impact: Excessive network requests, UI blocking

Pattern 4.2: No Request Deduplication

- Detection: Same endpoint called multiple times in parallel, no caching
- Impact: N duplicate network requests

Pattern 4.3: Infinite Re-render Loops

- Detection: useEffect updates state in its own dependency array
 - Impact: Browser freeze, crash
-

4.6 Category 5: Memory Leaks & Cleanup (3 Patterns)

Pattern 5.1: Missing Cleanup in useEffect

- Detection: useEffect with subscriptions/timers, no return function
- Impact: Memory leaks, eventual crash

Pattern 5.2: Event Listeners Not Removed

- Detection: addEventListener in useEffect, no removeEventListener in cleanup
- Impact: Memory accumulation over time

Pattern 5.3: Large State Objects Not Cleared

- Detection: State variables >1MB, data not cleared on unmount
 - Impact: Memory usage grows unbounded
-

4.7 Category 6: Performance API Misuse (2 Patterns)

Pattern 6.1: Blocking Main Thread

- Detection: Expensive calculations in render (not in useMemo)
- Impact: UI freeze, poor FID

Pattern 6.2: Missing Virtualization for Long Lists

- Detection: Array with >100 items rendered, no react-window
 - Impact: 10x slower rendering for lists >1000 items
-

4.8 Detection Implementation Strategy

Static Analysis Approach:

1. Parse TypeScript/JavaScript files using @typescript-eslint/parser.
2. Traverse AST looking for pattern signatures.
3. Count occurrences and severity.
4. Generate feature vector for ML model.

Git Diff Analysis:

1. Compare before/after to identify new anti-patterns introduced.
2. Track anti-pattern density over time.
3. Correlate with performance metric changes.

5. Feature Engineering Methodology

The feature engineering methodology defines 28 features to be extracted from each commit for ML model training.

5.1 Feature Categories Overview

Category	Features	Purpose
Static Code Metrics	8	Quantify code change magnitude
React-Specific Patterns	12	Detect framework-specific issues
Historical Context	8	Incorporate temporal patterns
TOTAL	28	Comprehensive performance signal

5.2 Static Code Metrics (8 Features)

F1: files_changed

- Type: Integer
- Definition: Number of files modified in commit
- Extraction: `git diff-tree --no-commit-id --name-only -r <commit>`
- Rationale: Large changes increase regression risk

F2: files_in_components

- Type: Integer
- Definition: Files changed in /components/ directory
- Extraction: Filter files_changed by path pattern
- Rationale: Component changes directly affect rendering

F3: lines_added

- Type: Integer
- Definition: Total lines of code added
- Extraction: `git show --shortstat`, parse insertions
- Rationale: Large additions indicate new functionality

F4: lines_deleted

- Type: Integer
- Definition: Total lines of code removed
- Extraction: git show --shortstat, parse deletions

F5: net_lines

- Type: Integer
- Definition: lines_added - lines_deleted
- Extraction: Calculated field

F6: large_commit

- Type: Boolean (0/1)
- Definition: 1 if lines_added > 200, else 0
- Rationale: Large commits have higher regression probability

F7: package_json_changed

- Type: Boolean (0/1)
- Definition: 1 if package.json modified
- Rationale: Dependency changes can introduce heavy libraries

F8: complexity_delta

- Type: Float
- Definition: Change in cyclomatic complexity
- Rationale: Increased complexity correlates with bugs

5.3 React-Specific Pattern Features (12 Features)

F9: useEffect_added

- Type: Integer
- Definition: Count of new useEffect hooks introduced
- Extraction: AST parse, count useEffect(in added lines)
- Rationale: More effects = more side effect complexity

F10: useState_added

- Type: Integer
- Definition: Count of new useState hooks
- Rationale: More state = more re-render triggers

F11: useCallback_added

- Type: Integer
- Rationale: Indicates optimization awareness (positive signal)

F12: useMemo_added

- Type: Integer
- Rationale: Memoization reduces re-computation (positive)

F13: useContext_added

- Type: Integer
- Rationale: Context can cause wide re-renders if misused

F14: useReducer_added

- Type: Integer
- Rationale: Complex state management

F15: memo_added

- Type: Integer
- Definition: Count of React.memo wrappings added
- Rationale: Optimization applied (positive signal)

F16: lazy_added

- Type: Integer
- Definition: Count of React.lazy for code splitting
- Rationale: Code splitting reduces bundle (positive)

F17: inline_arrow_functions

- Type: Integer
- Definition: Count of inline arrow functions in JSX
- Extraction: Count onClick={() => patterns • Rationale: Breaks memoization (negative signal)

F18: console_log_added

- Type: Integer
- Rationale: Debug code left in, indicates rushed changes

F19: import_count_delta

- Type: Integer

- Definition: Change in number of import statements
- Rationale: More imports may indicate heavier dependencies

F20: nested_components

- Type: Integer
- Definition: Count of component definitions inside components
- Rationale: Causes unnecessary re-instantiation

5.4 Historical Context Features (8 Features)

F21: previous_commit_performance

- Type: Float
- Definition: Performance score of previous commit (0-100)
- Extraction: Query database for previous commit's Lighthouse score

F22: performance_trend_7d

- Type: Float
- Definition: Average performance change over last 7 days
- Extraction: Query metrics, compute moving average

F23: time_since_last_regression

- Type: Integer (days)
- Definition: Days since last identified regression
- Rationale: Recent regressions may indicate systemic issues

F24: deployment_frequency

- Type: Integer
- Definition: Number of commits in last 7 days
- Rationale: High frequency may correlate with less thorough review

F25: bundle_size_delta

- Type: Float (percentage)
- Definition: % change in bundle size from previous commit
- Rationale: Bundle size directly impacts load time

F26: component_count_delta

- Type: Integer

- Definition: Change in number of components

F27: hook_density

- Type: Float
- Definition: Total hooks / total components in commit
- Rationale: High density may indicate complex components

F28: optimization_ratio

- Type: Float
- Definition: $(memo_added + lazy_added) / (useEffect_added + useState_added)$
- Rationale: Indicates developer awareness of optimization

5.5 Feature Extraction Workflow

Step-by-Step Process:

1. Receive Commit Hash from GitHub webhook.
2. Fetch Commit Metadata using git log.
3. Get Code Diff using git show.
4. Parse Changed Files. (filter for .tsx, .ts, .jsx, .js)
5. AST Parsing for each changed file
6. Compute Static Metrics from git statistics.
7. Query Historical Data from database.
8. Calculate Derived Features
9. Construct Feature Vector [F1, F2, ..., F28]
10. Store in Database
11. Trigger ML Inference

Implementation Plan:

- Use TypeScript for feature extraction service.
- Leverage @typescript-eslint/parser for AST analysis.
- Cache parsed ASTs for performance.
- Parallel processing for multiple files.

5.6 Data Labeling Strategy

Ground Truth Source: Lighthouse CI Performance Scores

Labeling Process:

1. For each commit, checkout code at that commit hash
2. Run `npm install && npm run build`
3. Serve build locally using HTTP server
4. Run Lighthouse CLI in headless mode
5. Extract performance score (0-100) from JSON report
6. Compare with previous commit's score
7. Label as regression if: $(\text{current_score} - \text{previous_score}) / \text{previous_score} < -0.20$

Regression Definition:

- Regression: Performance score drops >20% compared to previous commit
- No Regression: Performance score stable or improves

Expected Distribution:

- ~10-15% regression rate
- Creates class imbalance requiring SMOTE or weighted loss

6. Repository Selection Strategy

Repository selection is critical for creating a diverse, representative dataset.

6.1 Selection Criteria

Criterion 1: Technology Stack

- React 16.8+ (Hooks support)
- TypeScript preferred
- Modern build tools (webpack, vite, parcel)

Criterion 2: Activity Level

- Active development (commits within last 6 months)
- Regular releases
- >100 total commits

Criterion 3: Code Quality

- Clean commit history
- Semantic commit conventions
- Code reviews visible in PRs

Criterion 4: Application Diversity

- Mix of types: admin panels, dashboards, CMS, e-commerce, productivity
- Range of sizes: small (<50K LOC), medium (50-150K), large (>150K)
- Different state management: Context, Redux, Zustand, MobX

Criterion 5: Build Availability

- Working build scripts in package.json
- Runnable locally for Lighthouse testing
- Minimal external dependencies

Criterion 6: License

- Open source with permissive license (MIT, Apache 2.0)

6.2 Selected Repositories

Small Applications (10-30 pages, <50K LOC):

1. Joplin (Note-taking Application)
 - URL: github.com/laurent22/joplin
 - Stars: 40K+
 - Tech: React + TypeScript
 - Size: ~18,000 LOC
 - Rationale: Desktop app, moderate complexity
2. Excalidraw (Drawing Tool)
 - URL: github.com/excalidraw/excalidraw
 - Stars: 70K+
 - Tech: React + TypeScript
 - Size: ~25,000 LOC
 - Rationale: Canvas-heavy, performance-critical

Medium Applications (30-100 pages, 50-150K LOC):

3. React Admin (Admin Framework)
 - URL: github.com/marmelab/react-admin
 - Stars: 24K+
 - Size: ~85,000 LOC
 - Rationale: CRUD operations, enterprise use case
4. Strapi Admin (CMS Admin Panel)
 - URL: github.com/strapi/strapi
 - Stars: 60K+
 - Size: ~92,000 LOC

- Rationale: Complex forms, state management

5. Discourse (Forum Software Frontend)

- URL: github.com/discourse/discourse
- Stars: 40K+
- Size: ~105,000 LOC
- Rationale: Real-time updates, community software

Large Applications (100+ pages, >150K LOC):

6. Grafana (Monitoring Dashboard)

- URL: github.com/grafana/grafana
- Stars: 60K+
- Size: ~520,000 LOC
- Rationale: Data visualization, performance-critical

7. Ant Design Pro (Enterprise Admin Template)

- URL: github.com/ant-design/ant-design-pro
- Stars: 35K+
- Size: ~180,000 LOC
- Rationale: Enterprise-scale, extensive component library

8. Metabase (Business Intelligence Frontend)

- URL: github.com/metabase/metabase
- Stars: 36K+
- Size: ~310,000 LOC
- Rationale: Complex data queries, interactive visualizations

6.3 Expected Dataset Composition

Target Commit Distribution:

Repository Size	Repos	Commits/Repo	Total Commits	Expected Regressions
Small	2	100-120	220	30 (14%)
Medium	3	100-110	320	45 (14%)
Large	3	90-110	300	42 (14%)
TOTAL	8	~105 avg	840	117 (14%)

Expected Class Distribution:

- No Regression: ~720 commits (86%)
- Regression: ~120 commits (14%)
- Class imbalance ratio: ~6:1

6.4 Data Collection Plan

Phase 1: Repository Cloning (Week 1)

- Clone all 8 repositories locally
- Verify build scripts work
- Document build commands

Phase 2: Commit Extraction (Week 1-2)

- Extract last 100-120 commits per repository
- Filter commits modifying React files
- Exclude merge commits and automated commits

Phase 3: Performance Measurement (Week 2-3)

- For each commit: checkout, install, build, run Lighthouse
- Extract performance score
- Store results in database

Phase 4: Labeling (Week 3)

- Compare performance scores between consecutive commits
- Label commits with >20% degradation as regressions
- Validate sample manually

Phase 5: Feature Extraction (Week 4)

- Run feature extraction pipeline on all commits
- Generate 28-feature vectors
- Store features in database

Estimated Time:

- Cloning & setup: 2 hours
 - Commit extraction: 4 hours
 - Lighthouse automation: 8 hours (+ 24-48hr runtime)
 - Feature extraction: 12 hours (+ 6-8hr runtime)
 - Total: ~26 hours manual + ~30-56hr compute time
-

7. Technical Specifications

7.1 Development Environment

Hardware Requirements:

- CPU: Quad-core processor (Intel i5/i7 or equivalent)
- RAM: 16GB minimum (32GB recommended)
- Storage: 50GB free space
- Network: Stable internet connection

Software Requirements:

- OS: macOS, Linux, or Windows with WSL2
- Node.js: Version 18.x or higher
- Python: Version 3.9 or higher
- Git: Version 2.30 or higher
- PostgreSQL: Version 14.x or higher
- Docker: Version 20.x or higher

Development Tools:

- IDE: VS Code with extensions (ESLint, Prettier, Python, TypeScript)
 - Version Control: Git with GitHub account
 - API Testing: Postman or Insomnia
 - Database Client: pgAdmin or DBeaver
-

7.2 Technology Stack Details

Frontend (Monitoring SDK):

- react: ^18.2.0
- typescript: ^5.0.0
- vite: ^4.3.0
- @typescript-eslint/parser: ^5.59.0

Backend (API Service):

- express: ^4.18.0
- pg: ^8.11.0
- cors: ^2.8.5
- dotenv: ^16.0.0

- typescript: ^5.0.0

ML Service (Python):

- fastapi==0.104.0
- uvicorn==0.24.0
- xgboost==2.0.0
- scikit-learn==1.3.0
- pandas==2.1.0
- numpy==1.25.0

7.3 Database Schema

PostgreSQL Schema:

```
CREATE TABLE projects ( id UUID PRIMARY KEY DEFAULT gen_random_uuid(), name
VARCHAR(255) NOT NULL, github_url VARCHAR(500) NOT NULL, api_key VARCHAR(64)
UNIQUE NOT NULL, created_at TIMESTAMP DEFAULT CURRENT_TIMESTAMP );
```

```
CREATE TABLE commits ( id UUID PRIMARY KEY DEFAULT gen_random_uuid(),
project_id UUID REFERENCES projects(id), hash VARCHAR(40) UNIQUE NOT NULL,
message TEXT, author VARCHAR(255), timestamp TIMESTAMP NOT NULL, features
JSONB, prediction_score FLOAT, risk_level VARCHAR(20), is_regression BOOLEAN
);
```

```
CREATE TABLE metrics ( id UUID PRIMARY KEY DEFAULT gen_random_uuid(),
commit_id UUID REFERENCES commits(id), metric_type VARCHAR(50) NOT NULL,
value FLOAT NOT NULL, timestamp TIMESTAMP DEFAULT CURRENT_TIMESTAMP );
```

```
CREATE TABLE regressions ( id UUID PRIMARY KEY DEFAULT gen_random_uuid(),
commit_id UUID REFERENCES commits(id), severity VARCHAR(20) NOT NULL,
detected_at TIMESTAMP DEFAULT CURRENT_TIMESTAMP, resolved BOOLEAN DEFAULT
false );
```

7.4 Performance Requirements

Monitoring SDK:

- Bundle size: <50KB gzipped
- Performance overhead: <1% on host application
- Metric collection frequency: Every 30 seconds

Feature Extraction:

- Processing time: <10 seconds per commit
- Throughput: >100 commits/hour
- Memory usage: <2GB peak

ML Inference:

- Prediction latency: <500ms per commit
- Throughput: >1000 predictions/hour

Dashboard:

- Initial load time: <2 seconds
 - Time to Interactive: <3 seconds
 - Lighthouse Performance Score: >90
-

8. Implementation Plan

8.1 Month 2: Data Collection & Feature Engineering (March-April 2026)

Phase 2.1: Environment Setup (Week 1)

Tasks:

- ☐ Install all required software
- ☐ Set up project repository
- ☐ Configure PostgreSQL database
- ☐ Clone all 8 target repositories
- ☐ Verify build scripts
- ☐ Set up Lighthouse CI

Deliverables: Working development environment, repositories cloned Estimated Time: 8-10 hours

Phase 2.2: Commit Extraction (Week 2)

Tasks:

- ☐ Develop commit extraction script
- ☐ Extract 100-120 commits per repository (~840 total)
- ☐ Filter commits modifying React files
- ☐ Store commit metadata in database
- ☐ Run Lighthouse CI on each commit
- ☐ Label commits as regression/no-regression

Deliverables: ~840 commits with Lighthouse scores and labels Estimated Time: 16-20 hours + 30-48 hours compute

Phase 2.3: Feature Extraction Pipeline (Week 3)

Tasks:

- ☐ Implement AST parser
- ☐ Develop feature extraction functions for 28 features
- ☐ Create anti-pattern detection logic

- ☐ Build git diff analysis module
- ☐ Test pipeline on sample commits
- ☐ Run full extraction on entire dataset

Deliverables: Feature extraction pipeline, 28 features per commit Estimated Time: 20-24 hours

Phase 2.4: Dataset Preparation (Week 4)

Tasks:

- ☐ Export dataset to CSV/JSON
- ☐ Perform exploratory data analysis
- ☐ Validate feature distributions
- ☐ Split dataset (80% train, 20% test)
- ☐ Document dataset statistics

Deliverables: Clean dataset, EDA report, train/test split Estimated Time: 8-10 hours

Month 2 Total Time: 52-64 hours + computing time

8.2 Month 3: Model Training & System Implementation (April-May 2026)

Phase 3.1: Baseline Model Training (Week 1)

Tasks:

- ☐ Implement data preprocessing pipeline
- ☐ Handle class imbalance using SMOTE
- ☐ Train Logistic Regression baseline model
- ☐ Evaluate on test set
- ☐ Generate confusion matrix and ROC curve

Target Performance: Establish baseline metrics Deliverables: Trained baseline model, evaluation metrics Estimated Time: 8-10 hours

Phase 3.2: XGBoost Model Training (Week 1-2)

Tasks:

- ☐ Train initial XGBoost model

- ☐ Perform hyperparameter tuning using GridSearchCV
- ☐ Cross-validation across repositories
- ☐ Feature importance analysis
- ☐ Compare with baseline
- ☐ Save final model

Target Performance: Accuracy >75%, Precision >70%, Recall >60% Deliverables: Trained XGBoost model, comparison report Estimated Time: 12-16 hours

Phase 3.3: Monitoring SDK Development (Week 2)

Tasks:

- ☐ Create React package structure
- ☐ Implement Web Vitals collection
- ☐ Develop metric batching and transmission
- ☐ Build React Profiler integration
- ☐ Write unit tests
- ☐ Create NPM package

Deliverables: PerfSense SDK (npm package), documentation Estimated Time: 16-20 hours

Phase 3.4: Backend API Development (Week 3)

Tasks:

- ☐ Set up Express.js project
- ☐ Implement authentication middleware
- ☐ Create metrics ingestion endpoint
- ☐ Build project management endpoints
- ☐ Develop GitHub webhook handler
- ☐ Connect to PostgreSQL
- ☐ Deploy to cloud

Deliverables: Backend API service, API documentation Estimated Time: 16-20 hours

Phase 3.5: ML Inference Service (Week 3)

Tasks:

- ☐ Create FastAPI project

- ☐ Implement model loading logic
- ☐ Build prediction endpoint
- ☐ Add feature preprocessing
- ☐ Generate recommendations
- ☐ Containerize with Docker
- ☐ Deploy service

Deliverables: ML inference service, Docker image Estimated Time: 12-16 hours

Phase 3.6: Dashboard Development (Week 4)

Tasks:

- ☐ Set up React + TypeScript project
- ☐ Implement authentication
- ☐ Build commit analysis view
- ☐ Create performance visualizations
- ☐ Develop real-time metrics dashboard
- ☐ Style with TailwindCSS
- ☐ Deploy dashboard

Deliverables: Web dashboard application, deployed URL Estimated Time: 20-24 hours

Month 3 Total Time: 84-106 hours

8.3 Month 4: Evaluation & Documentation (May 2026)

Phase 4.1: System Integration & Testing (Week 1)

Tasks:

- ☐ Integrate all components
- ☐ Test end-to-end workflow
- ☐ Fix integration bugs
- ☐ Performance testing
- ☐ Security review

Deliverables: Fully integrated system, test results Estimated Time: 12-16 hours

Phase 4.2: Real-World Case Studies (Week 1-2)

Tasks:

- ☐ Build 2-3 demo React applications
- ☐ Integrate PerfSense SDK
- ☐ Introduce deliberate performance regressions
- ☐ Validate system predictions
- ☐ Document case study results

Deliverables: 2-3 demo applications, case study reports Estimated Time: 16-20 hours

Phase 4.3: Final Evaluation (Week 2)

Tasks:

- ☐ Evaluate models on held-out test set
- ☐ Generate final performance metrics
- ☐ Create comprehensive visualizations
- ☐ Conduct ablation study
- ☐ Document limitations

Deliverables: Final evaluation report, visualizations Estimated Time: 12-16 hours

Phase 4.4: Dissertation Writing (Week 3-4)

Tasks:

- ☐ Write all dissertation sections (80-100 pages)
- ☐ Create figures and tables
- ☐ Format references
- ☐ Proofread and edit
- ☐ Prepare presentation slides

Deliverables: Complete dissertation, presentation slides Estimated Time: 40-50 hours

Month 4 Total Time: 80-102 hours

Total Project Time Estimate: 216-272 hours over 4 months

9. Challenges and Mitigation Strategies

9.1 Technical Challenges

Challenge 1: Lighthouse CI Execution Time Problem:

- Running Lighthouse on 800+ commits may take 24-48 hours Mitigation:
- Parallel execution using multiple machines.
- Start with subset (200 commits) to validate approach.
- Use bundle size as quick proxy.
- Cache Lighthouse results Contingency: If too slow, use bundle size + manual labeling

Challenge 2: Repository Build Failures Problem:

- Historical commits may not build due to dependency changes Mitigation
- Error handling and logging
- Use npm ci with package-lock.json
- Focus on recent commits (last 6-12 months)
- Manual fixes for important commits Contingency: Accept 10-20% build failure rate

Challenge 3: Feature Extraction Complexity Problem:

- AST parsing may be complex to implement correctly Mitigation
- Incremental development (simple features first)
- Unit testing for each feature function
- Manual verification on sample commits
- Leverage existing ESLint rules Contingency
- Use regex-based pattern matching if AST too difficult

Challenge 4: Class Imbalance Problem

- Regression commits are rare (~10-15%) Mitigation
- SMOTE for synthetic oversampling
- `class_weight='balanced'` in sklearn
- Weighted loss in XGBoost (`scale_pos_weight`)
- Ensemble methods Contingency: Accept lower recall (>50%) if precision good

Challenge 5: Model Generalization Problem

- Model may overfit to training repositories Mitigation
- Cross-repository validation (train on 7, test on 1)
- Diverse dataset (wide range of types/sizes)

- Generalizable features (not repo-specific)
 - L1/L2 regularization Contingency: Document limitations, recommend project-specific fine-tuning
-

9.2 Time Management Challenges

Challenge 6: Underestimated Implementation Time Problem

- Development may take longer than 216-272 hours Mitigation
- Weekly tracking of actual vs. planned hours
- MVP approach (core features first)
- Parallel work where possible
- Built-in 20% buffer Contingency
- Reduce dashboard scope if behind by Week 10

Challenge 7: Work-Life Balance (WILP Student) Problem

- Balancing full-time job with 15-20 hours/week dissertation Mitigation
 - Fixed schedule (evenings 8-11pm, weekends 6 hours)
 - Keep supervisor informed.
 - Realistic weekly targets
 - Maintain health (sleep, exercise)
 - Contingency: Request extension if needed (communicate early)
-

9.3 Data Quality Challenges

Challenge 8: Inconsistent Commit Quality Problem

- Commits vary widely in quality Mitigation: Filter merge commits, automated commits, tiny commits
- Outlier detection in feature distributions
- Manual review of sample
- Multiple repositories reduce impact Contingency: Accept some noise, rely on ML to learn signal

Challenge 9: Labeling Accuracy Problem

- Lighthouse scores may not perfectly reflect user-perceived performance Mitigation
- Multiple metrics (bundle size, build time as secondary)
- Conservative threshold (>20% degradation)

- Manual validation of sample
 - Plan to refine labeling in future Contingency: Document limitations in dissertation.
-

9.4 Supervisor & Academic Challenges

Challenge 10: Meeting Supervisor Expectations Problem: Supervisor may have different expectations Mitigation:

- Bi-weekly progress updates
- Clear communication of timeline, challenges
- Documentation of decisions and rationale
- Flexibility to feedback Contingency: Negotiate timeline extension if major changes requested

Challenge 11: Academic Writing Quality Problem:

- Dissertation may not meet academic standards Mitigation
 - Write sections incrementally (not all at end)
 - Peer review of drafts
 - Use BITS dissertation template
 - Professional proofreading if needed Contingency: Allocate extra week for revisions in Month 4
-

10. Timeline and Next Steps

10.1 Detailed Timeline (Months 1-4)

Month	Weeks	Phase	Status	Completion
Month 1	Week 1-2	Literature Review	✓ Complete	100%
Month 1	Week 3-4	System Design & Repository Selection	✓ Complete	100%
Month 2	Week 1-2	Anti-Pattern Framework & Feature Design	✓ Complete	100%
Month 2	Week 3-4	Data Collection & Feature Extraction	🕒 Starting	0%
Month 3	Week 1-2	Model Training & Tuning	🕒 Planned	0%
Month 3	Week 3-4	SDK & Backend Development	🕒 Planned	0%
Month 4	Week 1-2	System Integration & Case Studies	🕒 Planned	0%
Month 4	Week 3-4	Final Evaluation & Dissertation Writing	🕒 Planned	0%

Overall Progress: Design Phase 100% Complete | Implementation Phase 0% | Evaluation Phase 0%

10.2 Immediate Next Steps (Next 2 Weeks)

Week 1 (April 15-21, 2026): Priority: Environment Setup & Data Collection Start

Day 1-2:

- ☐ Install all required software (Node.js, Python, PostgreSQL)
- ☐ Set up project folder structure
- ☐ Initialize Git repository ☐ Configure PostgreSQL database

Day 3-4:

- ☐ Clone all 8 target repositories
- ☐ Verify build scripts for each repository

- ☐ Document build commands
- ☐ Set up Lighthouse CI

Day 5-7:

- ☐ Develop commit extraction script
- ☐ Extract commits from first 2-3 repositories
- ☐ Test Lighthouse automation on sample commits
- ☐ Begin storing data in database

Target: 3 repositories analyzed, 200+ commits extracted

Week 2 (April 22-28, 2026): Priority: Complete Data Collection & Start Feature Extraction

Day 1-3:

- ☐ Extract commits from remaining 5 repositories
- ☐ Run Lighthouse CI on all commits (parallel execution)
- ☐ Label commits as regression/no-regression
- ☐ Validate dataset quality

Day 4-7:

- ☐ Begin implementing feature extraction pipeline
- ☐ Develop AST parsing module
- ☐ Extract features for first 100 commits
- ☐ Validate feature extraction accuracy

Target: 800+ commits collected and labeled, feature extraction started

10.3 Success Metrics

By End of Month 2 (April 30):

- ✓ 800+ commits collected from 8 repositories
- ✓ All commits labeled (regression/no-regression)
- ✓ Feature extraction pipeline implemented

✓ 28 features extracted for all commits

✓ Dataset ready for ML training.

By End of Month 3 (May 31):

✓ Baseline and XGBoost models trained

✓ Models achieve >75% accuracy, >70% precision

✓ Monitoring SDK built and assessed

✓ Backend API and ML service deployed

✓ Dashboard functional

By End of Month 4 (June 30):

✓ System fully integrated and evaluated

✓ 2-3 real-world case studies completed

✓ Final evaluation with comprehensive metrics

✓ 80-100 page dissertation written

✓ Presentation slides prepared

✓ Ready for viva defense

10.4 Risk Indicators

Red Flags (Immediate Action Required):

- Dataset <500 commits by end of Week 6
- Model accuracy <70% after tuning
- No working prototype by end of Month 3
- Dissertation not started by Week 14

Yellow Flags (Monitor Closely):

- Behind schedule by >1 week
- Build failures >30% of commits
- Feature extraction taking >20 seconds/commit
- Supervisor concerns about approach

Green Signals (On Track):

- Meeting weekly milestones
 - Dataset growing as planned
 - Regular code commits to GitHub
 - Positive supervisor feedback
-

10.5 Communication Plan

Supervisor Meetings:

- Frequency: Bi-weekly (every 2 weeks)
- Duration: 30-45 minutes
- Agenda: Progress update, challenges, decisions, next steps
- Format: Video call or in-person at Cadence office

Progress Documentation:

- Weekly Log: Brief summary of work, hours, blockers
- GitHub Commits: Regular commits with meaningful messages
- Monthly Report: Detailed progress report at end of each month

Escalation Protocol:

- Minor issues: Resolve independently
 - Moderate issues: Raise in next supervisor meeting
 - Major issues: Email supervisor immediately
-

11. Abbreviations

Abbreviation	Full Form
API	Application Programming Interface
APM	Application Performance Monitoring
AST	Abstract Syntax Tree
CI/CD	Continuous Integration/Continuous Deployment
CLS	Cumulative Layout Shift
CMS	Content Management System
CPU	Central Processing Unit
CRUD	Create, Read, Update, Delete
CSS	Cascading Style Sheets
DOM	Document Object Model
EDA	Exploratory Data Analysis
ESLint	ECMAScript Linter
FCP	First Contentful Paint
FID	First Input Delay
GPU	Graphics Processing Unit
HTTP	Hypertext Transfer Protocol
IDE	Integrated Development Environment

Abbreviation	Full Form
JSON	JavaScript Object Notation
JSX	JavaScript XML
LCP	Largest Contentful Paint
LOC	Lines of Code
ML	Machine Learning
MVP	Minimum Viable Product
NPM	Node Package Manager
PR	Pull Request
RAM	Random Access Memory
REST	Representational State Transfer
ROC-AUC	Receiver Operating Characteristic - Area Under Curve
SDK	Software Development Kit
SMOTE	Synthetic Minority Over-sampling Technique
SPA	Single Page Application
SQL	Structured Query Language
TTI	Time to Interactive
UI	User Interface
URL	Uniform Resource Locator
UX	User Experience
WILP	Work Integrated Learning Programme
XGBoost	Extreme Gradient Boosting

12. Conclusion

This mid-semester report documents the comprehensive design and planning work completed for the PerfSense dissertation project. The first two months focused on establishing a solid foundation through literature review, system architecture design, anti-pattern framework development, and detailed feature engineering methodology.

Key Achievements: ✓ Complete system architecture with 6 main components designed ✓ 20+ React anti-patterns identified and categorized ✓ 28-feature extraction methodology fully specified ✓ 8 target repositories selected spanning small to large applications ✓ Technology stack finalized (React/TypeScript, Node.js, Python, XGBoost, PostgreSQL) ✓ Implementation plan with detailed tasks and timelines.

Current Status: The project is transitioning from the design phase to the implementation phase. All architectural decisions have been made, and the technical approach has been validated through literature review and feasibility analysis. The project is on schedule with a clear roadmap for the remaining work.

Next Steps: The immediate focus is on data collection and feature extraction (Months 2-3), followed by model training, system implementation, and final evaluation (Months 3-4). With the detailed implementation plan in place and mitigation strategies for identified challenges, the project is well-positioned to achieve its objectives.

Expected Contributions: Upon completion, PerfSense will contribute:

1. Academic: First ML-based system for predicting React performance regressions pre-deployment
2. Practical: Actionable tool for developers to catch performance issues early
3. Dataset: Open source labeled dataset of React commits with performance metrics
4. Framework: Reusable anti-pattern detection framework for React applications

The project aims to achieve >75% accuracy, >70% precision, and >60% recall in predicting performance regressions, with a complete working system ready for evaluation and defense by June 2026.